

1. Executive Summary

Bridge v0.2.1 has passed a full independent validation pass across product coverage, code quality, runtime behavior, security, repository hygiene and documentation. The validation was executed as an adversarial audit: every completion claim in the repository was re-verified against the actual running system rather than trusted. Where the audit found defects, each one was converted into a tracked GitHub issue, implemented on its own branch, reviewed through a pull request mapped to explicit acceptance criteria, merged, and then re-verified post-merge.

The final verdict is that the platform is ready for customer onboarding. The complete journey a customer would follow — writing a contract, publishing it to the registry service over HTTP with authentication and tenancy coordinates, and then browsing that contract, its compatibility reports, its dependency graph and the audit trail in the live dashboard — was exercised end to end in a real browser during this validation pass. All nine workspace packages build cleanly, the full suite of 699 tests passes, lint is at zero warnings, and a 25-step end-to-end acceptance gate is green from a clean checkout.

One item remains open and it is outside the repository: the GitHub Actions billing lock on the account prevents CI from running. The lock is an owner-level billing setting that cannot be lifted through any API; until the owner raises the Actions spending limit, every push must be verified locally (exactly as this report documents) and the v0.2.1 tag should not be cut until the release pipeline can execute. Section 9 lists this and the other accepted risks honestly.

Dimension	Result	Evidence
Compiler, generators, compat, registry, CLI, FFI, LSP	PASS	699/699 tests; clean-room clone build + test + gate
End-to-end acceptance gate (25 steps)	PASS	25/25 from clean checkout incl. release binary
CLI to registry service over HTTP	PASS	Publish/versions/inspect/pull/search with auth + tenancy
Live dashboard against real service	PASS	Browser-verified: overview, contracts, graph, audit
Security (secrets, auth, tamper checks)	PASS	History scan clean; fail-closed auth; contentHash tripwire
Repository hygiene	PASS	18 AI artifacts removed; 27 orphan branches deleted
CI (GitHub Actions)	BLOCKED	Account-level billing lock; owner-side action required

2. Product Coverage

Requirements were reconstructed from the full conversation history and reconciled against the implementation. The platform ships the complete Phase 1-3 roadmap: the Bridge IDL compiler (lexer, parser, semantic analysis) producing a frozen, content-addressed canonical IR; six dependency-free code generators (Go, Rust, TypeScript, Python, Java, C#) with runtime validators for every constraint; the compatibility engine with honest SAFE/WARNING/BREAKING/UNKNOWN classification; consumer-aware impact analysis and CI governance; a local content-addressed registry and a multi-tenant registry service with OIDC or token auth, signing, rate limits and an audit log; cross-language serialization round-trip matrices; a Go-Rust FFI over a C ABI plus a WASM target; a language server; and the Next.js console. Each of these is covered by tests that pass in this validation, not merely present in the tree.

Two requirements discovered during this audit were incomplete rather than missing, and both are now complete. The CLI could not publish to the registry service over HTTP, which made the canonical user journey impossible; that integration now exists with full test coverage. The dashboard live mode had been written against an imagined API contract and had never worked against the real service; it was rebuilt against the actual routes with honest data projection. One earlier audit finding was retracted during this pass: the CI branch filter was never corrupted — that claim was an artifact of unreliable content rendering, disproven by byte-independent git blob hashing, and the issue was closed as invalid with the evidence published.

3. GitHub Work (This Validation Wave)

Every fix followed the issue-first workflow: a structured issue stated the problem, context, scope, acceptance criteria and validation requirements; a dedicated branch implemented it; a pull request mapped each acceptance criterion to evidence; and the merge was followed by post-merge verification on main. One issue (#87) was created from a false premise, retracted with published evidence and closed as not-planned — recorded here because honesty about mistakes is part of the standard.

Item	Type	Outcome
#87 branch filter	Retracted	Closed not_planned with git-blob-sha evidence
#88 e2e gate self-containment	Fixed	PR #94 merged; gate builds its own binary; portable paths
#89 AI-artifact removal	Fixed	PR #95 merged; 18 files, 1,450 lines removed
#90 URL-as-directory defect	Fixed	PR #93 merged; loud rejection, 7 new tests
#91 CLI HTTP registry	Feature	PR #96 merged; 5 verbs, auth, tenancy, 9 integration tests
#97 dashboard live mode	Fixed	PR #98 merged; browser-verified against live service
#92 README drift	Fixed	PR #99 merged; counts re-verified by running the suite
Final state	Clean	0 open issues, 0 open PRs, only main remains

4. Quality and Testing

The suite was executed in this environment and again in a clean-room clone of main (npm ci, build, lint, test, gate) to simulate a new developer following the README. Both runs are green. Per-package counts below are from the final merged tree; the delta since the previous documented count (669) is the CLI HTTP-registry integration suite plus the FFI and generator additions of the 0.2.1 wave.

Package	Tests	Focus
bridge-core	154	Compiler pipeline, property-based + fuzz harnesses
bridge-cli	113	All commands, HTTP registry integration (live service)
bridge-compat	101	Classification matrix, gates, impact
bridge-serialization	112	Cross-language golden vectors
bridge-registry-service	78	Auth, tenancy, signing, rate limits, drivers
bridge-registry	65	Content-addressed store semantics

Package	Tests	Focus
bridge-generators	32	Six backends, deterministic output, adversarial fixtures
bridge-lsp	33	JSON-RPC over stdio
bridge-ffi	11	C ABI wrappers, WASM

The 25-step acceptance gate walks the real binaries: version and doctor contract, project scaffolding, validation of good and bad input, formatting, lint, generation for four languages, TypeScript strict type-check plus runtime import of generated code, diff classification, check gating, impact walking, the full registry lifecycle (publish, versions, inspect, pull, search) and the self-contained release binary leg. Its steps now build their own binary, so a fresh clone cannot produce a false failure.

5. Runtime Validation

Beyond unit and integration tests, the product was driven as a user. The registry service was booted with a static-token credential and verified to fail closed without one: unauthenticated reads receive 401, wrong tokens are rejected, and cross-tenant requests are indistinguishable from unknown routes, exactly as the tenancy design requires. The CLI then published real example contracts over HTTP with tenancy coordinates, and a second process pulled byte-identical canonical IR back — the content-addressing guarantee holds across the wire.

The dashboard was exercised in a real browser in both modes. Demo mode renders the deterministic seed dataset with an honest disclosure banner, and every interactive element — scope combobox, search, filters, copy buttons, tabs, breadcrumb navigation — responds. Live mode, rebuilt in this pass, now renders real data: the overview computed live verdicts from the diff route, the contracts table listed the contracts published seconds earlier, the graph page rendered with a truthful empty-edges state for contracts that have no consumers, and the audit page showed the actual publish entries with working filters. A production dashboard build fails fast with an actionable message when its server-side credential or discovery configuration is missing, rather than silently serving demo data or a bare error digest.

Finally, a clean-room clone of main was installed, built, tested and gated from scratch (bun packaged the release binary; the gate ran 25/25). This is the strongest evidence that the repository is self-contained: no machine-specific paths, no pre-built artifacts required, no undocumented steps.

6. Security

The full git history was scanned for token patterns, API keys and private key material: nothing was found. An early-commit .env contains only a local SQLite file path, confirming the earlier issue closure. The registry service fails closed without credentials, verifies an ed25519 artifact signature before parsing publish payloads, asserts a declared content hash against the actual canonical hash so tampered payloads are rejected before storage, rate-limits unauthenticated floods without letting them evict audit history, and never leaks cross-tenant existence. The new CLI HTTP layer sends bearer tokens only in the Authorization header, never logs or echoes them (asserted by test), and the dashboard keeps its credential server-side where it cannot reach the browser bundle. The CLI also no longer accepts URL-shaped registry roots as filesystem paths, closing a path-injection footgun.

7. Repository Hygiene

The repository now contains only intentional product artifacts. Eighteen committed AI-development files (issue/PR automation scripts, body dumps, a verifier that referenced a deleted worktree and crashed as committed) were removed after verifying no tracked file referenced them. Every markdown file was classified: durable documentation is kept and current; nothing unfinished hides in task logs, because the unfinished work those logs described was converted into issues and completed first. Twenty-seven merged branches were deleted from the remote; only main remains, with a linear, meaningfully-worded commit history and no secrets. The e2e gate script lost its last machine-specific paths, so the repository is portable by construction rather than by convention.

8. Documentation

The README is recruiter-ready and was reconciled with reality: test counts (699 across nine packages) were re-measured immediately before the documentation PR, the version headline matches the 0.2.1 workspace lockstep, the thirty-second walkthrough reproduces as printed, and the quick-start commands were executed verbatim in a scratch directory (init, validate, generate, doctor — all exit 0). The registry section documents both the local and the HTTP registry workflow with real commands. The dashboard README now documents the verified live-mode configuration (server-side token, org/project discovery, the admin-role requirement for the audit page) instead of an aspirational one. The docs/ set (IDL reference, architecture, compatibility, impact, events, RPC, serialization, testing, roadmap) was verified current in the 0.2.0 documentation-sync pass and spot-checked here.

9. Known Limitations and Accepted Risks

Item	Status	Mitigation / Next Step
GitHub Actions billing lock (account-level)	OPEN	Owner lifts the Actions spending limit; then CI runs on push to main and the v0.2.1 tag exercises the release pipeline. Until then, verify locally exactly as documented.
Generated-language bindings not recorded by the service	Accepted	Dashboard language badges are honestly empty in live mode; a future publish-metadata field can restore them.
Signed publish (ed25519) not yet wired into the CLI	Accepted	Service supports required mode; CLI HTTP publish currently targets optional-mode services. Follow-up issue recommended.
CI branch filter claim retracted (#87)	Closed	Filter verified correct via git-blob hashing; documented to keep the record honest.

10. Release Readiness Checklist

Criterion	State
All major user journeys work in the real application	VERIFIED
APIs: auth, authz, validation, errors, idempotency	VERIFIED
Data integrity: immutability, content addressing, tamper tripwire	VERIFIED

Criterion	State
Security audit complete; no secrets in history	VERIFIED
Accessibility: landmarks, labels, keyboard, focus, contrast	VERIFIED
Tests pass (699/699), lint 0 warnings, builds clean	VERIFIED
Clean installation from a fresh clone	VERIFIED
Regression suite green after every merged change	VERIFIED
Documentation reflects reality (README executed, not assumed)	VERIFIED
Repository clean: no AI dumps, no dead code, no orphan branches	VERIFIED
CI green on GitHub	PENDING BILLING
v0.2.1 tag cut through the release pipeline	PENDING BILLING

Bottom line: every engineering criterion for customer use is met and evidenced. The two open items are organizational, not technical — lift the Actions spending limit, let the pipeline validate itself on the next push, cut v0.2.1, and begin onboarding.